Ders 14 Genişletilmiş Notu: Büyük Dil Modelleri

Ders 14 Genişletilmiş Notu: Büyük Dil Modelleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı
- 2. Dil Modelleri Nasıl Eğitilir?
- 3. Eğitim Kümesinde Sıfır Hata Mümkün mü?
- 4. Amaçlanmadan Ortaya Çıkan Kabiliyetler
- 5. Fine-Tune ve X-Shot
- 6. GPT-3 Sonuçları ve Sistematik Olmayan Hatalar
- 7. Çoktan Seçmeli Sorular ve İlginç Bulgular
- 8. OpenAI API ve Gelecek Çalışma
- 9. Görevden Bağımsız Yapı, Hizalama ve InstructGPT
- 10. Pekiştirmeli Öğrenme Temelleri
- 11. Dil Modellerini RL ile Fine-Tune Etmek: PPO
- 12. Hazır Tercih Verileri ve Açık Kaynak Çalışmalar
- 13. ChatGPT ve OpenAssistant
- 14. Henüz Yapamadıkları: Plan Yapma
- 15. Tartışma - 2023
- Örnek Sorular
- Dersin Toparlaması

1. Dersin Planı

Bu, dönemin anlatım içeren son dersiydi. Konu büyük dil modelleriydi (Large Language Models, LLM). Slayt dosyası “bolum8_LLM” adını taşıyor ve 27 slayttan oluşuyor (ekrandan, 00:20:00). Hoca konunun güncelliğine dikkat çekti: bu modeller popüler, başarılı ve gündemde; önümüzdeki dönemde ilginin azalacağı da görünmüyor.

İçerik slaytına göre plan dört parçaydı (ekrandan, 00:20:00):

- Otoregresif dil modelleri.
- Uyarlanmış (fine-tune edilmiş) büyük dil modelleri.
- Bu işlerde kullanılan yöntemler.
- Bir tartışma slaytı: nereye gidiyoruz?

2. Dil Modelleri Nasıl Eğitilir?

2.1 Eğiticili öğrenmenin maliyeti ve ucuz yol

Hoca önce eğiticili (supervised) öğrenmeyi hatırlattı: modele “böyle bir girdi gelirse böyle bir çıktı ver” denir, yani bir girdi-çıktı eşlemesi vardır (00:18:30). Çıktı kısmı insan gücüyle hazırlanıyorsa bu hiç ucuz değildir ve eğitim kümesinin büyüklüğünü sınırlar.

Daha ucuz bir yol vardır: elimizde bir metin var ve onun orijinal halini zaten biliyoruz. Metni bozarız ve modelden onu olması gereken haline döndürmesini isteriz (00:19:53). Bu birkaç biçimde yapılır:

- **Next:** “Şunlardan sonra ne gelir?” diye sorulursa bu next token prediction olur.
- **Masked:** Bir dizinin içindeki bir parça maskelenip ne olduğu sorulur.
- **Changed:** Bir şeyin içindeki bir parça değiştirilir (kurallarla ya da başka bir modelle) ve hangi parçanın değiştiği tahmin ettirilir.

Slayttaki ifade (ekrandan, 00:20:00): “Büyük dil modelleri (LLM) token tahmini ile eğitilirler (next, masked, changed). Artık oto-regresif modeller (next thing prediction) standart. Çünkü yeni görevlere çok daha kolay uyum sağlıyor.”

2.2 Neden “next thing”?

Hoca slayta bilerek “thing” yazdığını açıkladı: tahmin edilen şey kelime, token, harf ya da cümle olabilir (00:20:59). Ölçek değişebilir, ama iş hep bir sonrakini tahmin etmektir. Bu şu an büyük dil modelleri için standart gibi görünüyor; daha önce ağırlıklı olarak maskeli token tahminiyle model üreten firmalar bile next token prediction’a geçmiş durumda.

Bunun temel sebebi yeni görevlere kolay uyumdur. Önceden eğitilmiş (pre-trained) modelleri eğitmenin amacı tek başına bir problemi çözmek değildir; onları elimizdeki başka problemleri çözmek için kullanırız. Her türlü görevi “bundan sonra ne gelir?” kapsamında ifade etmenin yolları bulunabildiği için otoregresif yapılar yeni görevlere çok daha kolay uyum sağlar (00:22:11).

2.3 Kısa tarihçe

BERT temelde maskeli bir yapıdır ve popülerliği maskeli dil modellerine ilgiyi epey artırdı. GPT’nin Transformer’lı hali BERT’ten sonra çıktı (00:23:16). Ama next token prediction hiç de yeni bir yaklaşım değildir. Hoca, yanlış hatırlamıyorsa, geri yayılımın (backpropagation) önerildiği makalede bile küçük bir sonraki kelime tahmini deneyi olduğunu söyledi (00:23:40).

Konunun popüler hale gelmesi Kasım-Aralık 2022’de ChatGPT ile büyük dil modellerinin neler yapabildiğinin herkese açık biçimde gösterilmesiyle başladı. Hoca o tarihten bu yana yaklaşık altı ay geçtiğini ve alanın çok hızlı ilerlediğini söyledi (00:24:03).

3. Eğitim Kümesinde Sıfır Hata Mümkün mü?

3.1 Ezberleme ve dilin doğası

Hoca şu soruyu sordu: next token ya da maskeli tahminle eğitilen bir model eğitim kümesinde sıfır hataya erişebilir mi, yani eğitim kümesini ezberleyebilir mi? (00:24:37)

Elinizde büyük bir model ve küçük bir veri kümesi varsa model normalde ezberleyebilir; teknik olarak bir sorun yoktur. Ama dil verisinde ezberleyememenin bir sebebi vardır. Hoca bunu bir örnekle açıkladı: slayttaki “LLM’ler eğitim kümesinde” ifadesini düşünün ve devamını görmediğinizi varsayın (00:26:10). Bundan sonra çok farklı şeyler yazılabilir ve gerçekten de yazılıyor. İnternette “LLM’ler eğitim kümesinde” diye başlayan cümleleri ararsanız epey cümle bulursunuz ve her biri farklı biçimde devam eder.

Klasik makine öğrenmesi açısından bu, aynı noktada farklı sınıflardan örnekler olması demektir (00:27:43). Aynı başlangıçla başlayıp farklı devam eden çok sayıda cümle varsa her biri için doğru cevabı üretmek mümkün değildir. Bu yüzden yeterince büyük bir eğitim kümesinde sıfır hataya erişilemez.

3.2 Hatayı azaltmak temsilleri yakınlaştırır

Sıfır hata mümkün değil, ama model hatayı azaltmaya çalışıyor. Bunun için aynı girdiden sonra gelebilecek farklı çıktıların iç temsillerini birbirine yaklaştırması gerekir ki bazen onu, bazen bunu tahmin edebilsin (00:28:46). Bu temsilleri birbirinden alakasız yerlere götürürse hatası çok daha fazla olur.

Slayttaki soru bu fikri anlam temsillerine bağlar (ekrandan, 00:30:00): “LLM’ler eğitim kümesinde 0 hataya erişebilirler mi? Düşük hataya erişebilmeleri için benzer anlama/kullanıma sahip kelimelerin temsilleri ?”

3.3 Dil modeli bir sınıflandırıcıdır

Hoca daha önceki haftalarda da konuşulan bir noktayı tekrarladı: dil modelleri regresyon değil sınıflandırma yapar (00:29:48). Her adımda o anki duruma göre bir token seçerler. Sözlük 128 bin tokensa, softmax ile 128 bin sınıftan biri seçilir. Bu açıdan bakınca "aynı noktada farklı sınıflar" benzetmesi daha iyi oturur: "eğitim kümesinde" ifadesinden sonra gelebilecek her farklı kelime ayrı bir sınıftır ve aynı girdiye karşılık farklı çıktılar vardır. Model hatayı azaltmak için bu farklı devamların temsillerini benzer noktalara indirger (00:30:56).

4. Amaçlanmadan Ortaya Çıkan Kabiliyetler

Slaytın başlığı "Amaçlanmadan ortaya çıkan kabiliyetler - Emergent properties" (ekrandan, 00:32:10). Hoca Türkçe karşılığın kısa olmadığını ama daha iyisini bulamadığını söyledi; bir tür yan ürün gibi düşünülebilir. Asıl amaç en başta böyle bir şey olması değildir.

Slayttaki maddeler:

- "Autoregressive LLM's are ???": slayttaki makale başlığı "Language Models are Few-Shot Learners" (OpenAI'ın GPT-3 makalesi).
- "Yeni bir görev için modelde güncelleme / finetune yapmadan sonuç alabilmek."

Yapılan iş temelde bir sonraki kelimeyi tahmin etmektir. Ama bu modellerin en başta amaçlanmayan bazı kabiliyetlere sahip olduğu görülür: yeni bir görev için modeli güncellemeden, fine-tune yapmadan sonuç alabilmek (00:32:16). Ön eğitimin amacı bir problemi çözmek değil, başka problemleri çözerken yardımcı olacak sağlam bir altyapı kurmaktır. Model ve veri büyüklükleri belli sınırları aştıktan sonra bu modellerin hiç fine-tune edilmeden, olduğu haliyle belli türden problemleri çözebildiği görülmüştür (00:33:10).

5. Fine-Tune ve X-Shot

5.1 Geleneksel fine-tuning

Slaytın sol tarafı GPT-3 makalesinden alınmıştır: "Traditional fine-tuning (not used for GPT-3)" (ekrandan, 00:40:00). Hoca GPT-4'ün bu anlamda teknik bir makalesinin olmadığını da belirtti (00:33:38).

Fine-tune'da elimizde "şöyle bir girdiden sonra şöyle bir çıktı tahmin etmen lazım" diyen bir veri kümesi vardır. Modele girdiyi veririz, model bir çıktı söyler. Söylediği şeyin olması gerekenden farkına göre kaybı ve kaybın gradyanlarını hesaplar, ağırlıkları güncelleriz (00:34:20). Önceden eğitilmiş model bir tarafta durur; yeni görev için girdi-çıktı ikililerinden oluşan yeni bir veri kümesiyle modelin eğitimine devam edilir. Eğitim öncekine göre çok daha kısa sürer ve çok daha küçük bir veriyle yapılır, ama ağırlıklar değişir (00:35:03).

Slayttaki şema: "sea otter => loutre de mer" (örnek 1) → gradient update → "peppermint => menthe poivrée" (örnek 2) → gradient update → … → "plush girafe => girafe peluche" (örnek N) → gradient update → "cheese =>" (istem) (ekrandan, 00:40:00).

Hocanın örneği: görev, cümlenin son kelimesini tahmin etmek olsun (00:35:15). Böyle bir görev için next token prediction ile eğitilmiş modeli bir daha fine-tune etmenin pek anlamı yoktur. Cümlenin son kelimesini silip modele verdiğimizde, model next token olarak bir şey tahmin eder ve görevin beklediği de zaten budur. Eğitim mantığı ön eğitimle birebir aynıdır.

5.2 Zero-, one- ve few-shot

X-shot'ta bir tür koşullandırma vardır, ama bu bir fine-tune süreci değildir (00:36:33). Slayttaki üç ayar şöyledir (ekrandan, 00:40:00):

- **Zero-shot:** Model yalnızca görevin doğal dildeki tanımını görür; gradyan güncellemesi yapılmaz.

  ```
  Translate English to French:     <- görev tanımı (task description)
  cheese =>                         <- istem (prompt)
  ```

- **One-shot:** Görev tanımına ek olarak tek bir örnek görür.

```
Translate English to French:
sea otter => loutre de mer        <- örnek
cheese =>
```

- **Few-shot:** Birkaç örnek görür.

```
Translate English to French:
sea otter => loutre de mer
peppermint => menthe poivrée
plush girafe => girafe peluche
cheese =>
```

Hoca zero-shot'ı şöyle açıkladı: "Translate English to French:" yazıp ardından "cheese" ve bir işaret koyarsınız. İşaretin tam olarak bu olması şart değildir. Sonra next token prediction çalışır: yazılan dizinin devamında ne gelecek? (00:36:44) Görev tanımını koymadan yalnızca "cheese" yazarsanız devamı çok daha muğlak olur; görev tanımı modelin ne demek istediğinizi anlaması için ek bilgi verir. Ama arka tarafta görev tanımıyla eğitilmiş bir şey yoktur; en azından bize söylenen, modelin yalnızca next token prediction ile eğitildiğidir (00:37:44).

One-shot'ta "şunu dedim, buna karşı bunu söyle" diye bir örnek verilir; few-shot'ta İngilizceden Fransızcaya üç örnek verip ardından "cheese" sorulur (00:38:45).

> **Hocanın vurgusu:** Bu üçü de next token prediction ile eğitilmiş modele doğrudan verilir. Bu noktada hiçbir eğitim yoktur (00:38:58).

5.3 LAMBADA sonuçları

Slayttaki grafik LAMBADA görevi üzerindedir (ekrandan, 00:40:00). Slayttaki açıklama: "Lambada görevi: Cümlenin son kelimesini tahmin, uzun bağımlılıkları bulma kabiliyeti."

- X ekseni dil modelinin milyar cinsinden parametre sayısıdır: 0.1B, 0.4B, 0.8B, 1.3B, 2.6B, 6.7B, 13B, 175B. 175B GPT-3'ün kendisidir; ötekiler aynı veri üzerinde eğitilmiş daha az parametreli sürümleridir (00:39:23).
- Y ekseni doğruluktur (accuracy). Grafikte "Human" ve "Zero-Shot SOTA" düzeyleri de çizilidir.
- 0.1B, yani 100 milyon parametreli modelin zero-shot doğruluğu %40 civarındadır (00:39:48).

Hocanın gözlemleri (00:40:29):

- Parametre sayısı arttıkça üç ayar da giderek iyileşir.
- Few-shot (K=15) belli bir noktadan sonra ötekileri aşar; bu biraz beklenen bir durumdur.
- One-shot bazen zero-shot'tan kötüdür: bazı noktalarda yeşil (one-shot) aşağıda, bazılarında yukarıdadır. Sistematik bir düzen yoktur (00:41:13).
- Buradaki modellerin hiçbiri fine-tune edilmemiştir.

Hoca few-shot örnek sayısının görevden göreve değiştiğini, LAMBADA'da K=15 olduğunu söyledi (00:41:59). Aritmetik görevlerinde de soru doğrudan sorulmaz; öncesinde birkaç örnek verilir.

6. GPT-3 Sonuçları ve Sistematik Olmayan Hatalar

6.1 Başka görevler

"Başka?" başlıklı slaytta dört grafik vardır (ekrandan, 00:42:30):

- **Arithmetic (few-shot):** İki, üç, dört ve beş basamaklı toplama ve çıkarma, iki basamaklı çarpma, tek basamaklı üç işlem. Küçük modellerde doğruluk sıfıra yakındır; 13B'den sonra özellikle iki ve üç basamaklı toplama-çıkarmada hızla yükselir.
- **Translation (Multi-BLEU):** Fransızca, Almanca, Romence ile İngilizce arasında iki yönlü çeviri; parametre arttıkça BLEU artar.
- **Wordscramble (few-shot):** Harf döndürme (cycle letters), anagramlar, rastgele ekleme, ters yazılmış kelimeler.
- **İnsanların model tarafından üretilen haberi ayırt etme yeteneği:** Parametre sayısı arttıkça düşer.

Son görevde modelden belli bir konuda bir metin yazması istenir. Bu metin ve insanların yazdığı metin yine insanların karşısına konur ve “bunu insan mı yazdı, makine mi?” diye sorulur (00:42:44). Başlangıçta model küçükken insanların doğruluğu %75 civarındaydı; parametre arttıkça %50’ye düştü (00:43:07). %50 kabaca ayırt edilemediği anlamına gelir: metni bir insan mı, GPT-3 mü yazdı, bilinemiyor.

Çeviride erişilen BLEU skorları, eğiticili, yani fine-tune edilmiş yöntemlerle yarışabilecek düzeydedir (00:43:39). Bu sonuçlar bir görevi baştan vermeden, fine-tune yapmadan, ağırlıklarda hiçbir değişiklik yapmadan yalnızca akıllıca tasarlanmış girdilerle alınmıştır. Buradan görevden bağımsız (task agnostic), her türlü görev için çalışabilecek yapılar fikrine gidildi (00:44:35).

6.2 Sistematik olmayan hatalar

Hoca grafiklerde bir sistematik olmadığını tekrar vurguladı (00:44:48). Bize göre çok daha zor bazı işlerde iyi sonuç alınırken daha kolaylarında işler iyi gitmeyebiliyor. Bir kabiliyet kümesi pek görünmüyor; birçok görevde hâlâ kötü performans var.

GPT-4 için de benzer durumlar görülüyor. Mimarinin büyüklüğü hakkında çok bilgi yok, ama hâlâ sistematik olmayan hatalar var. Hocanın örneği: model 40 basamaklı iki sayıyı toplayabiliyor, ama bu sayılardan birini 35 basamaklı yaptığınızda toplayamıyor (00:46:25). Bu, beklentimizle hiç uyuşmayan garip bir durum.

> **Hocanın vurgusu:** Bir insan size “6 basamaklı sayıları çarpabiliyorum ama 3 basamaklıları çarpamıyorum” ya da “çarpmayı çok iyi yapıyorum ama toplamayı beceremiyorum” dese bunu kafanızda oturtamazsınız. Bu bizim alışık olduğumuz bir durum değil (00:46:56).

7. Çoktan Seçmeli Sorular ve İlginç Bulgular

7.1 Çoktan seçmeli sorular nasıl cevaplatılır?

Slaytın başlığı: “Çoktan seçmeli sorular içeren veri kümelerinde X-shot: Her cevabın üretilme olasılığı bulunup, büyük olan seçiliyor” (ekrandan, 00:50:00).

Soru doğrudan sorulabilir ya da “Soru şu, cevap şu” biçimine dönüştürülebilir (00:47:43). Slayttaki örnekler:

Veri kümesi	Biçim	Girdi	Seçenekler
CommonsenseQA	Minimal	What blocks sunshine? \n	summer, park, desktop, sea, moon
CommonsenseQA	Manual	The question is: What blocks sunshine? \n The answer is:	summer, park, desktop, sea, moon
COPA	Minimal	Effect: I coughed. \n	Cause: I inhaled smoke. / Cause: I lowered my voice.
COPA	Manual	I coughed because	I inhaled smoke. / I lowered my voice.
ARC	Minimal	Which biome has the most vegetation? \n	desert, forest, grassland, tundra
ARC	Manual	The question is: Which biome has the most vegetation? \n The answer is:	desert, forest, grassland, tundra

Modelden “cevabı üret, seçeneklerden biriyse doğrudur” diye beklenmez; bu hem çok zor hem de pek anlamlı değildir (00:48:32). Bunun yerine her seçenek için “şu ifadenin devamında bunun gelmesi” olasılığı hesaplanır. Dil modelleri bir cümlenin olasılığını hesaplayabilir: “The

answer is:” sonrasına “summer” koyarsanız cümlenin bir olasılığı, “park” koyarsanız başka bir olasılığı, “desktop” koyarsanız yine başka bir olasılığı vardır (00:49:00). Beş olasılıktan hangisi en yüksekse modelin cevabı o kabul edilir. Birçok benchmark veri kümesi çoktan seçmeli olduğu için bu yaklaşım yaygındır.

7.2 Cevaplar yanlış olsa da

Slaytın üst kısmı (ekrandan, 00:52:10) bir duygu sınıflandırma örneğidir. Few-shot örnekleri (sarı) ve sorulan cümle (yeşil) birlikte girdiyi oluşturur:

```
Circulation revenue has increased by 5% in Finland.  \n Positive
Panostaja did not disclose the purchase price.       \n Neutral
Paying off the national debt will be extremely painful. \n Negative
The company anticipated its operating profit to improve. \n ____
-> LM -> Positive (Correct!)
```

Hoca burada da bir eğitim olmadığını vurguladı: “Böyle bir şeye karşı böyle bir şey de” demeye çalışıyoruz ama bunu eğitimde değil girdide yapıyoruz; havada bir veri kümesi oluşturuyoruz (00:50:42).

İlginç olan şudur: örneklerdeki etiketler karıştırılırsa, yani rastgele verilirse, sorulan cümleye doğru cevap verme oranı çok fazla düşmüyor (00:51:37). Slayttaki ikinci örnekte aynı üç cümleye “Neutral, Negative, Positive” verilmiş ve model yine “Positive” diyerek doğru cevaplamış. Slayttaki çubuk grafiklerde sarılar doğru etiketli, kırmızılar rastgele etiketli durumun performansıdır (Classification ve Multi-choice; GPT-2, MetaICL, GPT-J, fairseq 6.7B ve 13B, GPT-3 modelleri; Direct ve Channel yöntemleri). Normalde büyük bir düşüş beklenir, ama öyle olmuyor (00:52:04).

Hoca bunu şöyle yorumladı: girdide modele “bunu dersen bunu de” diye çok açık bir şey söylemiyoruz. Açıkça söylediğimiz yer fine-tune ya da ön eğitim kısmıdır (00:52:28).

Öte yandan etiketler bambaşka, alakasız kelimelerle değiştirilince performans gidiyor; model bu sefer anlamlı bir şey vermiyor (00:53:47). Yani etiketlerin kendi içinde karışması çok etkilemiyor.

Soru: Örnekler yanlış etiketli ama hâlâ bir kombinasyon gibi veriliyor. Hepsi negatif ya da hepsi pozitif verilse ne olur, bu denenmiş mi? (00:54:10)

Cevap: Hoca önce bunu etiketleri bambaşka kelimelerle değiştirmekten ayırdı. Örneğin “pozitife bundan sonra armut, nötre çöp kutusu, negatife çerçeve diyeceğim” derseniz model pozitif bir cümleye “armut” demeli; bu da yapılabiliyor ve ayrı bir konu (00:54:39). Buradaki bulgu ise etiketlerin kendi içinde karışmasıdır: gerçekte negatif olan bir cümleye “pozitif” denmişse tutarlı olmak için başka bir yerde de negatife “pozitif” demek gerekirdi, ama yine de model doğru cevabı veriyor (00:55:10). Bütün örneklerin aynı etiketle verildiği durumla ilgili bir çalışma görmediğini söyledi. Ona göre bu durumda modelin negatif bir cümleye “negatif” demesi pek mümkün görünmüyor, çünkü model örneklerde söylenen etiketlerden birini söyler, onların dışında bir şey söylemez (00:56:31).

7.3 Hiç görmediği kelimeler olsa da

Aynı slaytın alt kısmında model, tanımı verilen uydurma kelimeleri cümlede kullanır (ekrandan, 00:53:10):

```
A "Burringo" is a car with very fast acceleration. An example of a sentence
that uses the word Burringo is:
In our garage we have a Burringo that my father drives to work every day.

A "Gigamuru" is a type of Japanese musical instrument. An example of a
sentence that uses the word Gigamuru is:
I have a Gigamuru that my uncle gave me as a gift. I love to play it at home.

To "screeg" something is to swing a sword at it. An example of a sentence
that uses the word screeg is:
```

```
We screeghed at each other for several minutes and then we went outside
and ate ice cream.
```

7.4 Bu nasıl mümkün oluyor?

Slayt 9'un başlığı "Bu nasıl mümkün oluyor?"; maddesi "Net bir açıklama yok şimdilik" ve bir Stanford blog yazısına bağlantı (ekrandan, 00:57:40, kenar çubuğu). Hoca, özellikle etiketler karıştırıldığında bile modelin anlamlı bir şey üretebilmesine dair Bayesçi bir bakış açısından bir açıklama girişimi olduğunu söyledi (00:53:16). Ama çok net bir açıklama yok.

> **Ek bilgi (derste anlatılmadı):** Bağlam içi öğrenmenin (in-context learning) Bayesçi yorumunda, ön eğitim sırasında model metinlerin altında yatan gizli "konuları" ya da görevleri çıkarmayı öğrenir. Few-shot örnekleri modele hangi gizli görevin söz konusu olduğunu buldurur; etiketlerin doğruluğundan çok girdi dağılımı, etiket uzayı ve biçim önemlidir. Bu, etiketler karıştırıldığında başarının az düşmesini büyük olasılıkla açıklar.

8. OpenAI API ve Gelecek Çalışma

8.1 Completion API'si

OpenAI API'sinin yapısı "Completion"dır: verilen metnin devamını tamamla (00:57:23). Bütün iş budur. Aynı API yapısıyla çok farklı görevler, sanki farklı görevlermiş gibi çözülebilir. Slayttaki kod (ekrandan, 00:57:40):

```python
import os
import openai
openai.api_key = os.getenv("OPENAI_API_KEY")
response = openai.Completion.create(
    model="text-davinci-003",
    prompt="The following is a list of companies and the categories they fall into:\n\nApple,
    temperature=0,
    max_tokens=64,
    top_p=1.0,
    frequency_penalty=0.0,
    presence_penalty=0.0
)
```

Slayttaki örnek cevap:

```
Technology

Facebook
Category: Social Media

Fedex
Category: Delivery
```

Hoca bunun bir araştırma alanına dönüştüğünü söyledi: bu modellerin içinden neler çıkarabiliriz? Doğru ya da bazen yanlış, çok ilginç sonuçlar elde edilebiliyor (00:58:54).

8.2 Future work

GPT-3 makalesi (2020) modeli fine-tune etmediğini açıkça söyler. Slayttaki alıntı (ekrandan, 01:10:00): "GPT-3 could also in principle be evaluated in the traditional fine-tuning setting, but we leave this to future work." Hocaya göre bu gelecek çalışma bugün gördüğümüz fine-tune edilmiş dil modelleridir (00:59:25).

9. Görevden Bağımsız Yapı, Hizalama ve InstructGPT

9.1 Görevden bağımsız yapı

Aradan sonra hoca GPT-3 makalesindeki gelecek çalışmanın nasıl yapıldığını anlattı. Slayttaki soru: "Bunu daha genel bir yapıda (task agnostic - görev bağımsız) yaparsak neler olur?" (ekrandan, 01:10:00)

Fikir şudur: farklı veri kümelerini ve farklı görevlerin hepsini birden ortak bir yapıda eğitelim, hepsini aynı biçimde verebilelim; makine çevirisini de, özetlemeyi de (01:10:24). Bunu yaparken insan dilini kullanalım. Slayttaki ifade: "İstediğimizi, metinle ifade edebilmek sınırsız bir ifade gücü veriyor bize, tabi model yeterli ise." İnsan diliyle bir şey söyleyebiliyorsak sisteme her şeyi söyleyebiliriz.

Bunun için eğitim süreci değiştirildi. Model fine-tune edilecek, ama fine-tune self-supervised (next token) yapıdan insan geri bildiriminden pekiştirmeli öğrenmeye (Reinforcement Learning from Human Feedback, RLHF) dönüştü (01:11:23). Slayt: "Self supervised → RLHF." Bunun ilk örneği ChatGPT değil, InstructGPT'dir (01:11:45).

9.2 Hizalama (alignment)

Slayt 13'ün maddeleri (ekrandan, 01:12:40):

- "Alignment: Modelin istediğimizi, **bizim!** kabul edebileceğimiz bir yolla yapması."
- "İstediğimizi (tüm dünya bilgisine sahip bir insan gibi cevap ver, ama **zararlı!** olabilecek çıktılar üretme) objective fonksiyon olarak yazmak zor, ama örneklerini verebiliriz."
- "Biz kimiz? Kime göre zararlı?"

Hoca bu amacı matematiksel bir amaç fonksiyonu olarak yazmanın zor, ama örneklerini vermenin mümkün olduğunu söyledi (01:12:37). "Zararlı" kime göre zararlıdır? "İnsanlığa" deseniz bile "hangi insanlara?" sorusuna dönüşebilir. Problem kolayca başka bir şeye dönüştürülebilir ve bu konuda şu sıralar çok tartışma yürüyor (01:12:48).

9.3 InstructGPT'nin adımları

InstructGPT bir diyalog modeli değildir: kullanıcı bir şey söyler, model bir cevap verir, sonrası yoktur (01:13:35). Amaç, bu tek girdi-tek çıktı sürecini görevden bağımsız bir yapıya büründürmek ve mümkün olduğunca çok görevde başarılı olduğunu göstermektir.

Slayttaki algoritma (ekrandan, 01:20:40):

1. Model 1: girdi-çıktı eğiticili eğitim (supervised GPT-3 fine-tuning).
2. Veri kümesi A: girdi ve Model 1'in farklı çıktıları arasında insanların tercihleri (comparison data).
3. Model 2 (Reward Model): veri kümesi A üzerinde eğitim; insanların verdikleri bilgilerle, insanların ödül verme mekanizmasını modeller.
4. Model 1, Model 2'nin ödül tahminleri kullanılarak RL ile eğitilir; 2. adıma dönülür.

Slayttaki not: "1. ve 3. adımlarda insan devrede. İlkinde cevap yazıyor, sonrakinde seçiyor. Toplamda 20 bin saat insanlar etkileşime girmiş."

Hocanın anlatımıyla adımlar şöyle işler:

- Elimizde GPT-3 var; onu model 0 diye düşünebiliriz. Önce eğiticili biçimde fine-tune edilir: insanlar belli girdilere bakıp "böyle bir girdiye böyle bir cevap yazmak gerekir" diye cevap yazar. Bu Model 1'dir (01:14:46).
- Model 1'e bir girdi verilir ve farklı çıktıları insanlara gösterilir. Gerçek insanlar "bu çıktı şundan daha iyi, bu bundan kötü" diye geri bildirim verir; çıktıları sıralar. Böylece veri kümesi A oluşur: "şu girdiye şu çıktıyı üretirsen insanlar iyi ya da kötü dedi" (01:15:13).
- Veri kümesi A ile yeni bir model eğitilir: ödül modeli (reward model), Model 2. Bu model insanların değerlendirmesini taklit eder: "şu girdiye şöyle bir çıktı üretirsen iyi mi yaptın, kötü mü?" sorusunu tahmin eder (01:16:20).
- Model 1, Model 2'nin ödül tahminleriyle, yeni bir istem kümesi üzerinde pekiştirmeli öğrenmeyle eğitilir (01:17:03).

- Güncellenen Model 1 farklı girdiler için farklı çıktılar üretmeye başlar; bu çıktılar için yine insanlardan tercih alınıp ödül modeli yeniden oluşturulur. Böylece 2. ve 4. adımlar arasında bir döngü sürer (01:19:37).

Hoca 20 bin saatin büyük olasılıkla 20 bin kişiyle birer saat olmadığını, insanların bu işte çalıştırıldığını söyledi (01:20:15).

9.4 Ödül modeli neden gerekli?

Ödül modelinin amacı ölçeklenebilirliktir (01:17:32). RLHF insanlardan geri bildirim almak demektir, ama insanları işin içine kattığınızda süreç ölçeklenebilir olmaz. Eğiticili yapıdan öz denetimli (self-supervised) yapıya geçişin temel motivasyonu da insanlardan bir şey alma ihtiyacını azaltmaktı (01:18:03). Burada da insanlardan belli örnekler toplanıp insanların yaptığı işi modelleyen bir ödül modeli kurulur. Ödül modeli iyi kurulursa insan aradan çıkar ve eğitim insansız yapılabilir (01:18:27). En uzun süren kısım 4. adımdır: çok sayıda örnek için çok sayıda eğitim yapılır. Bunu mümkün kılmak için ödül modeline ihtiyaç vardır (01:18:55).

9.5 Şekil üzerinden RLHF ve istem veri kümesi

Slayt 15'teki şekil (Reinforcement Learning from Human Feedback, OpenAI'ın ChatGPT sayfasından) aynı yapının şekilli halidir. İnsanlar iki yerde devreye girer: eğiticili fine-tuning için cevap yazarken ve çıktıları sıralarken (01:20:40). Üçüncü adımda yeni bir istem gelir, model bir şey üretir; üretilen şey ve girdi ödül modeline verilir; ödül modeli "bu girdiye bu çıktıyı ürettiysen iyi ya da kötü yaptın" diye bir ödül hesaplar. Bu ödüle göre pekiştirmeli öğrenme algoritmasıyla, burada PPO ile, modelin ağırlıkları güncellenir; yani model yine fine-tune edilir (01:22:11).

İstem veri kümesi (prompt dataset) şudur: OpenAI GPT-3'ü yayınladıktan sonra API'sine erişimi ücretli hale getirdi. İnsanlar fine-tune olmadan modelin içinden doğru bilgiyi almak için istemler yazmaya başladı (01:23:30). Bu istemler oldukça değerlidir; modelin yapmasını istediğimiz şeyleri gösterirler. Hocaya göre bütün iş bir istem mühendisliği (prompt engineering) yapısına dönüşüyor.

10. Pekiştirmeli Öğrenme Temelleri

10.1 RL nedir, neden kullanılır?

Hoca pekiştirmeli öğrenmenin (Reinforcement Learning, RL) bütün yapısına girmeyeceğini, yalnız gerekeni anlatacağını söyledi (01:24:24). Slayt 16 (ekrandan, 01:25:30):

- "Belli bir durumda ne yapması gerektiğini değil, yaptığının ne kadar iyi/kötü olduğunu geri bildirim olarak vermek."
- "Neden?" — "Biz de doğrudan çok emin olmayabiliriz." — "Tek doğrunun olmayışı."
- "Expedition vs. Exploration ikilemi: Yemeği nerede yiyelim? Bildiğimiz en iyisinde mi? Yeni bir yerde mi?"

Eğiticili öğrenmede "şuna karşı şunu üret" denir; eğiticisiz öğrenmede bir çıktı yoktur. RL bunlardan biraz farklıdır: model bir şey yapar ya da söyler; doğrusunun ne olduğunu söylemezsiniz, ama ne kadar iyi ya da ne kadar kötü yaptığına dair geri bildirim verirsiniz (01:24:36). Bunun iki sebebi vardır: o durumda yapılması gerekeni biz de tam bilmiyor olabiliriz ya da tek bir doğru olmayabilir (01:25:14). Böylece "şu girdiye şu çıktıyı üret" yaklaşımının dışında üçüncü bir yol ortaya çıkar.

RL'nin ilk yapısında ajan (agent; RL jargonunda model) belli bir çevrede hareket ederken en çok ödülü toplamaya çalışır (01:26:19). Ajan başta çevreyi bilmez. Hareketler yapar; her hareketten sonra ona "bu iyi oldu", "kötü oldu" ya da "ne kâr ne zarar" gibi bir şey söylenir. Amaç ortamdaki cezalardan kaçınıp ödülleri toplamaktır.

10.2 Kullan ya da keşfet ikilemi

RL'de karşımıza kullan ya da keşfet ikilemi çıkar (01:27:24). "Kullan", o ana kadar bulduğun en iyi yoldan devam etmektir. "Keşfet", şimdiye kadar bu yolu kullandık ama belki daha iyisini

buluruz diye farklı şeyler denemektir. Bu ikilem yalnız RL'nin değil, gündelik hayatın da ikilemidir. Slayttaki örnek: öğle yemeğine nereye gidelim, bildiğimiz en iyi yere mi, yeni bir yere mi? İkisinin de avantajı ve dezavantajı vardır (01:28:24). RL'de genelde başta keşif olasılığı yüksek tutulur ve zamanla yavaş yavaş azaltılır (01:28:41).

> **Ders sonrası not:** Slaytta ve anlatımda ikilemin "kullan" tarafı "expedition" diye geçiyor; doğru terim büyük olasılıkla "exploitation"dır (exploitation vs. exploration).

10.3 Policy, durum ve hareket

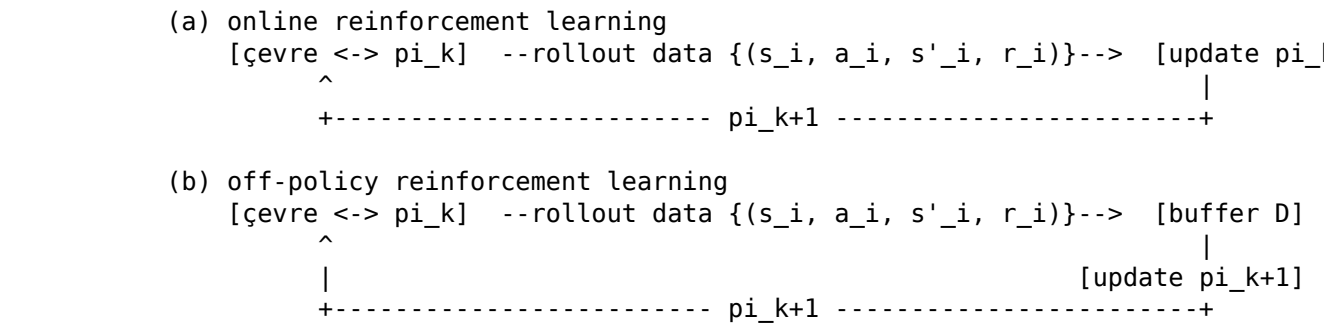
Slayt 17 "On / off policy / offline RL" (ekrandan, 01:30:00):

- Ajan, durumu (s_t) gözler, buna göre policy $\pi_\theta(a \mid s)$'i kullanıp hareket (a) seçer. Bunun sonucunda ödül (r) alır ve sonraki duruma (s_{t+1}) geçer.
- Policy: durumu harekete dönüştüren bir fonksiyon (amaç bunu optimize etmek).
- Toplanan deneyimlerin formatı: $\langle s, a, s', r \rangle$.
- Bu deneyimlerle policy eğitilir.
- Yöntemlerin farkı deneyimlerin üretim süreçleri.

Hoca bu kavramları dil modellerine eşledi (01:29:09). Robotlarda ya da oyun karakterlerinde hareket gerçekten fiziksel bir harekettir. Dil modellerinde ise üretilen şey harekettir: model A değil B der. Bir ajanın belli bir durumda dört hareketi (yukarı, aşağı, sağa, sola) olabilir; bir dil modelinin belli bir durumda örneğin 32 bin hareketi vardır ve bunlardan birini yapar (01:29:47). Bu, sınıflandırma benzetmesiyle de uyumludur. Durum, dil modelinde o anki temsil vektörü olarak düşünülebilir; policy dil modelinin kendisidir, θ da büyük dil modelinin parametreleridir (01:30:35).

10.4 Online, off-policy ve offline RL

Slayt 18'deki şema (ekrandan, 01:32:45):



- **Online RL:** Mevcut modelinize göre hareket eder, bu hareketlerden deneyim biriktirir ve doğrudan bunlarla eğitim yaparsınız (01:32:15). InstructGPT'deki de online RL'dir (01:32:42).
- **Off-policy RL:** Tek fark bir tamponlama (buffer) işidir. Tamponda modelin önceki halleri için üretilmiş deneyimler de bulunur ve bir sonraki güncellemede kullanılabilir (01:32:58). Tampon süreci daha kararlı yapar: son adımlarda iyi olmayan bir güncelleme modeli bozabilir; tampon bunu bir ölçüde engeller (01:33:35). Hoca off-policy yaklaşımların eğitimlerde yavaş yavaş kullanılmaya başladığını söyledi.
- **Offline RL:** Modelin farklı sürümlerinden kocaman bir veri kümesi üretip yeni bir modeli tamamen çevrimdışı eğitmek de mümkündür (01:34:05). Online eğitimde mevcut modelinizle hareketler yapar ve bunlara göre güncellersiniz; offline eğitimde yaptığınız hareketlerle bir ilginiz yoktur, yalnızca bir veri kümesindeki "şu durumda şunu yaparsan şunu görürsün" bilgisinden hareket edersiniz.

RLHF döngüsüne bağlarsak: güncellenmiş model yeni örnekler üretir, bunlardan yeni bir ödül modeli oluşturulur, ödül modeli yeniden geri bildirim vermeye başlar. Ödül modeli, şemadaki r'yi sağlayan yapıdır (01:35:04).

11. Dil Modellerini RL ile Fine-Tune Etmek: PPO

11.1 Gürültü ve momentum

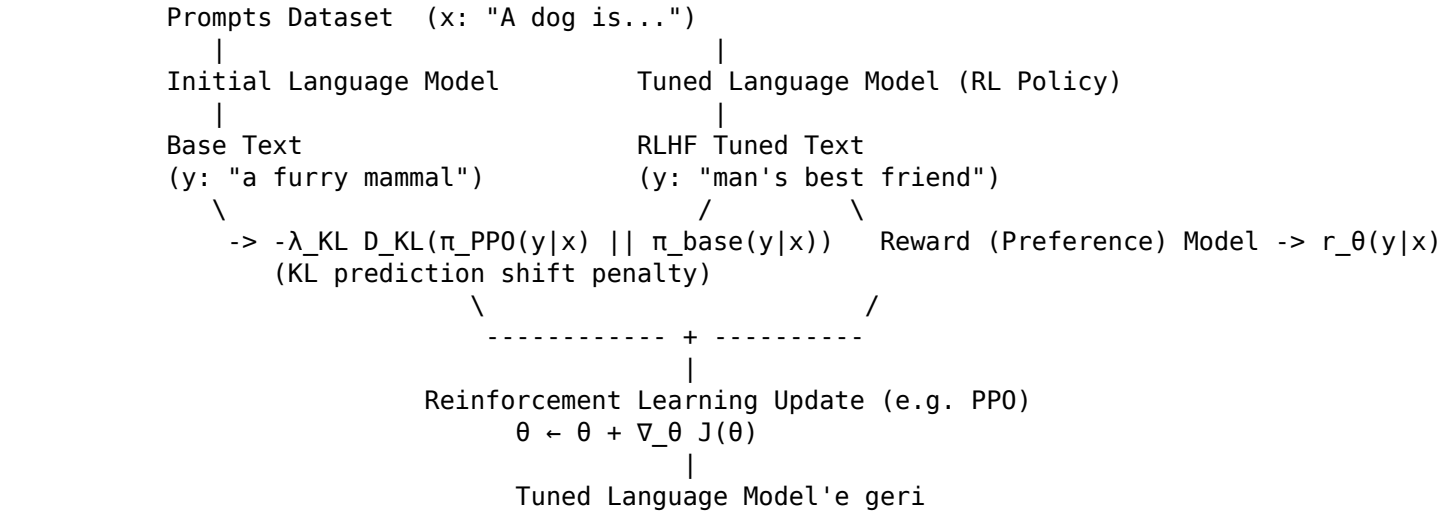
RL teknikleri temelde oyunlardan geliştiği için dil modellerine uyarlamak belli düşünceler gerektirir. Dil modelleri RL ile fine-tune edilirken PPO adlı bir RL algoritması kullanılır (01:35:30).

Hoca RL'nin gürültülü olduğunu stokastik gradyan inişiyle (SGD) karşılaştırarak anlattı (01:36:03). SGD'de bin örneğiniz varsa her güncellemede hepsinin gradyanını hesaplamak çok pahalıdır; bunun yerine rastgele bir örnek seçip onunla hareket edilir. Bunun mantığı, belli bir noktada farklı örneklerin üreteceği gradyanların birbirinden çok bağımsız olmamasıdır; özellikle sürecin başlarında oldukça ilişkilidirler (01:36:43). RL'de bu ilişki zayıftır, yani yapı gürültülüdür (01:37:11).

Gürültüyü engellemek için kullanılan temel yaklaşım momentumdur: önceki güncellemeyi de işin içine katar (01:37:32). Arkasında bir yığın bir şeyle geliyormuş gibidir: o anda gelen şeye göre doğrudan hareket etmezsiniz, önceki hareketiniz de şimdiki hareketinizde bir etkendir. PPO'da yapılan da buna biraz benzer: model güncellenirken eski halinin çıktılarından çok uzaklaşmak istenmez (01:37:56).

11.2 KL cezası

Slayt 19 "Dil modellerini RL ile finetune" (ekrandan, 01:40:00). Şemanın sol tarafı:



Sağ tarafta Rollout (Query "This movie is" → LM (GPT-2) → Response "really great!"), Evaluation (Query + Response → Reward model → Reward 1.0) ve Optimization (aktif model ve referans modelin log-olasılıkları → KL-div; Reward ile birlikte PPO) adımları vardır.

Ödüle eklenen ceza terimi şöyledir:

$$R(x, y) = r_\theta(y \mid x) - \lambda_{KL}\, D_{KL}(\pi_{PPO}(y \mid x) \,\|\, \pi_{base}(y \mid x))$$

Hoca kaybın iki parçası olduğunu söyledi (01:38:16). Bir parça KL ıraksamasıdır: iki policy'nin ürettiği çıktıların benzerliğine bakılır. Elimizde bir başlangıç modeli (initial model) ve fine-tune ettiğimiz yeni bir model var; yeni modelin eski modelden çok sapması istenmez.

Normalde RL için böyle iki modelli bir yapıya ihtiyaç yoktur: bir girdi geldiğinde doğrudan ödüle bakıp "istediğimizle uyumlu bir şey üretti mi?" diye güncelleme yapılabilir (01:38:34). Ama öyle yapılmıyor. Temel model, aslında modelin bir önceki sürümüdür: Model 1 eğitildikten sonra yeni bir sürüm oluşur, her sürüm bir sonrakinin temeli olur ve adım adım ilerlenir (01:39:30). Başlangıçtaki model istem veri kümesi açısından kötüdür, ama onun ürettiklerinden çok uzaklaşmak istenmez. Yapı yalnızca isteme özelleşirse ("şunu denince şunu de") dilin doğallığını, kalitesini büyük ölçüde kaybettiği görülmüş (01:40:25).

Yani model RL eğitiminde iki şey yapmaya çalışır: kullanıcının isteklerine cevap vermek ve bir önceki sürümden çok farklı çıktılar üretmemek (01:41:48).

Hocanın vurgusu: Bu yapıların hiçbiri bir gecede birinin aklına gelmedi; seneler boyunca adım adım geliştirildi. Yapay sinir ağları alanında her seçimin arkasında çok teori olmasa bile çok deney olduğundan emin olabilirsiniz (01:40:47).

Ek bilgi (derste anlatılmadı): PPO (Proximal Policy Optimization), Schulman ve arkadaşlarının büyük olasılıkla 2017'de önerdiği bir policy gradyan algoritmasıdır. Kendi içinde de yeni policy ile eski policy'nin olasılık oranını belli bir aralıkta kırparak (clipping) tek bir güncellemenin policy'yi çok değiştirmesini engeller. Slayttaki KL cezası ise ayrıca, RL ile ayarlanan modelin başlangıç dil modelinden uzaklaşmasını sınırlar.

12. Hazır Tercih Verileri ve Açık Kaynak Çalışmalar

12.1 Bazı veri kümeleri

Slayt 20 "Bazı Veri kümeleri" (ekrandan, 01:42:45, kenar çubuğu): Reddit "Explain Like I'm 5", BIG-bench, FLAN, Collective human preferences. Hoca OpenAI'ın bunları GPT-3 API'sine gelen istemler üzerinden yaptığını, ama açık kaynak topluluğun da bu konuda çok çalıştığını ve hazır birçok yapının bu işe kolayca dönüştürülebildiğini söyledi (01:42:13).

12.2 Topluluğun tercihleri

Slayt 21 "Topluluğun tercihleri - Collective human prefences" (ekrandan, 01:42:45):

- İnsan geri bildirimi veri kümelerini ucuza elde etmek.
- Forumlardan hazır veri toplamak.
- Bir soruya verilmiş birçok cevap içinde hangisinin daha iyi olduğunu belirlemek.
- Nasıl? Like ve vote sayılarına göre tercihleri otomatik toplamak (soru, cevap1, cevap2, hangisi iyi).
- Stanford human preferences dataset (SHP): iyilik için zamanı da kullanmak (daha çok oy alan ve eski olan daha iyi, çünkü yeni henüz yeterince görülmemiş olabilir).

Hoca bunu şöyle bağladı: forumlarda "şuna karşı şöyle bir cevap" vardır, ama bir şey daha vardır: oylar (01:42:50). Bir soruya verilmiş birçok cevaptan hangisinin daha iyi olduğunu oylar gösterir. Ödül modelini eğittiğimiz veri kümesi A da tam olarak buydu: bir girdiye karşı çıktılardan hangisinin daha iyi olduğu. Orada bunu insanlardan toplamaya çalıştık; forumlarda insanlar zaten soru sormuş, cevap yazmış ve puan vermiş. Bu bilgi otomatik olarak kullanılabilir.

- **Avantaj:** İnsanlara ekstra para harcamazsınız (01:43:46).
- **Dezavantaj:** İnsanların verdiği oylar her zaman "daha iyi"yi göstermez. "Kime göre iyi, kime göre zararlı" sorusu burada da vardır; insanlar bir cevabı başka nedenlerle beğenebilir. Bu, temel bilimlerden çok sosyal bilimler alanındaki konularda yaygındır (01:43:52).

  Hocanın vurgusu: Hazır veri kullanmak kesinlikle iyi fikir, ama bu veriler her zaman istediğimiz şeyi ifade etmiyor olabilir (01:44:12).

12.3 Açık modeller ve Alpaca

Büyük firmalar bu tür modelleri kapalı biçimde yayınlamaya başlayınca bazı firmalar açık biçimde yayınlamaya başladı; hoca Meta'nın yayınladığı LLaMA'yı örnek verdi (01:44:39). Açık dil modellerini sohbet kabiliyetine sahip, görevden bağımsız yapılara dönüştürmek için çok çalışma başladı. Hocaya göre neredeyse birkaç günde bir yepyeni sonuçlar ve yöntemler çıkıyor; literatürü takip etmek bile çok zorlaştı (01:45:05).

Hoca açıklık konusunda iki ucun çatıştığını söyledi: böyle güçlü bir şey olacaksa herkesin erişimine açık olması mantıklıdır; öte yandan bu atom bombası gibi bir şeyse yapım tarifini her yere koymamak gerekir. Ona göre tamamen kontrol altına almak artık olası görünmüyor (01:45:56).

Slayt 22 "Büyük Dil Modellerini Taklit (Alpaca)" (ekrandan, 01:47:30):

- "GPT3.5 (>175B)'yi kullanıp, self instruction'la LLaMA (7B)'yı eğitmek, <600$ (ek insan eforu yok)."

Şemadaki süreç:

```
Text-davinci-003 + 175 Self-Instruct seed tasks
   -> Modified Self-instruct Instruction Generation
   -> 52K Instruction-following examples
   -> (Meta LLaMA 7B ile) Supervised Finetuning
   -> Alpaca 7B
```

Slayttaki örnekler:

- Örnek tohum görev: “Instruction: Brainstorm a list of possible New Year’s resolutions. Output: Lose weight, Exercise more, Eat healthier.”
- Örnek üretilmiş görev: “Instruction: Brainstorm creative ideas for designing a conference room. Output: … incorporating flexible components, such as moveable walls and furniture …”
- GPT’den yeni görev üretim süreci: “Come up with a series of tasks: Task 1: {instruction for existing task 1} … Task 8: {instruction for existing task 8} Task 9:”

Hocanın anlatımıyla: GPT-3.5, talimat (instruction) veri kümesini üretmek için kullanıldı. Belli talimatlar ve onlara karşı cevaplar verilip “bunlara benzeyen başka örnekler bul” diye soruldu. Kendi oluşturdukları 175 görevlik bir set kullanılıp 52 bin örnek çıkarıldı (01:47:45).

Hoca bu 52 bin örneği kendisinin incelediğini ve çok çeşitli (diverse) bulduğunu söyledi (01:48:09). Örneğin “10+10=20”, “15+15=30” gibi yalnızca sayıyı artırmak için üretilmiş tekrarlar değil; gerçekten geniş kapsamlı. İyileri de kötüleri de var, ama çeşitlilik konusunda ikna olmuş. Alpaca, GPT’ye göre çok daha küçük bir yapıdır: 175 milyara karşı 7 milyar. Bu veri kümesiyle fine-tune edildiğinde hiç de fena olmayan sonuçlar verdiği gösterildi (01:49:02).

13. ChatGPT ve OpenAssistant

13.1 ChatGPT

Slayt 23 (ekrandan, 01:50:00):

- Ticari, yöntemi çok bilinmiyor. Bildiklerimiz:
  - InstructGPT’nin bir sürümü.
  - Diyalog tabanlı (yeni cevap, önceki soru ve cevaplara da bağlı).
  - Daha büyük model, daha çok veri ile eğitilmiş.
- Açık kaynak yeniden üretimi: OpenAssistant.

Hoca ChatGPT’nin yöntemine dair gerçek bir teknik makale olmadığını, ama amaç olarak muadili sayılabilecek birçok açık kaynak proje olduğunu söyledi; örnek olarak OpenAssistant’ı seçti (01:49:45).

13.2 OpenAssistant ve konuşma ağacı

Slayt 24 “OpenAssistant: Gönüllülerden veri toplama (Crowd-Sourced Training Data Collection)” (ekrandan, 01:51:30, kenar çubuğu). Hoca bu şemanın deminki InstructGPT şemasının biraz daha ayrıntılı yazılmış hali olduğunu, çok büyük bir fark olmadığını söyledi (01:50:10). Burada asıl soru, tek girdi-tek çıktı yapısını diyaloğa nasıl dönüştüreceğimizdir. Bunun için konuşma ağacı (conversation tree) kullanılır: kullanıcı bir şey dedi, sistem bir şey dedi, kullanıcı yine bir şey dedi, sistem bir şey dedi. Bütün iş bu ağaca bir şeyler eklemektir (01:50:28).

Slayt 25 “Nasıl veri toplayalım? Görevler” (ekrandan, 01:51:30):

- **Main Task 1: Reply to conversation.** Bir konuşmanın bir sonraki girişini önermek. Örnek konuşma: “Teach me Chess” → “Sure, Chess is an ancient game…” → “How do I cheat?” → (önerilen cevap) “I am a large…”.
- **Main Task 2: Label a text.** Bu metin küfür (profanity), alay (sarcasm), şiddet (violence) içeriyor mu? Bu metin yararlı mı (helpful), çok kısa mı (too short), çok belirsiz mi (too vague)?

- **Main Task 3: Rank replies.** “Teach me Chess” → “Sure, Chess is an ancient game…” → “What’s the best move?” sorusuna verilmiş “It depends, …”, “What is the game state?” ve “e4” cevaplarını kaliteye göre sıralamak (order all replies by quality).

Hocanın anlatımıyla: insanlara görevler verilir. Örneğin “şöyle bir sohbet var, devamını getirir misin?” denir: kullanıcı şunu demiş, model şunu demiş, kullanıcı şunu demiş; şimdi model ne desin? (01:51:09) Ya da cevapları sıralamaları istenir: kullanıcının bu sözüne model şu üç cevaptan birini verebilir; hangisini seçersiniz? Ya da modelin bir diyalogda söylediği şeye etiket verilir: zararlı, şiddet içeriyor, işe yarar, hiç işe yaramaz, çok uzun, çok kısa gibi (01:51:42).

ChatGPT’nin yapısının da böyle olduğu tahmin ediliyor: tek girdi-tek çıktı yapısının diyaloğa dönüştürülmesi ve insanların bu türden görevlerle çalıştırılması. Hoca bunun kesin olarak söylenemeyeceğini, yalnızca tahmin olduğunu vurguladı (01:52:10).

14. Henüz Yapamadıkları: Plan Yapma

Slayt 26 “Henüz” (ekrandan, 01:53:20):

- Plan yapma (kaynak: “Sparks of Artificial General Intelligence: Early experiments with GPT-4”).
- Tüm işlem tek bir çizgi üzerinde, her şey aynı zorluk seviyesinde.
- Nasıl aşılabilir? (bir openreview makalesine bağlantı)
- Aşmalı mıyız?
- Aşılabiliyorsa durdurabilir miyiz?

Bu modeller çok şey yapabiliyor, ama yapamadıkları da var; en azından plan yapma konusunda sorun var (01:52:49). Hoca Microsoft’un GPT-4 için yaptığı değerlendirmeyi slayta aldığını, Microsoft’un yaptığı bir değerlendirme olduğu için buna güvenilebileceğini söyledi (01:53:34).

Engel şudur: tüm işler tek bir çizgi üzerinde yürür; dinamik biçimde ileri geri gidilemez (01:53:45). Bunu aşmak için yaklaşımlar var: çıkmaz bir sokağa girersen sokağın başına geri dön gibi. Bunlara Chain of Thought, Tree of Thoughts gibi adlar veriliyor (01:54:21).

Hoca bu yaklaşımlara biraz şüpheyle baktığını söyledi (01:54:34). Model temelde soldan sağa ilerleyen bir yapıdır. İnsanda dinamik olduğunu düşündüğümüz süreç, aslında yine bu yapının içine giydirilmeye çalışılıyor. Mimaride de veride de asıl büyük kısım next token prediction ile önceden eğitilmiş yapıda durur; fine-tune bunun üstündeki kısımdır (01:54:47). Oraya dokunmadan, oradaki eğitim sürecini değiştirmeden yalnızca üst taraflarla oynayıp akıl yürütmeyi istediği gibi yapabilen bir yapı kurmak hocaya çok kolay gelmiyor; ama yanılıyor olabileceğini de ekledi (01:55:29).

15. Tartışma - 2023

Son slayt (ekrandan, 02:00:00):

- **Hinton:** “İnsan beyni kadar iyi çalışan sistemler yapmaya çalıştık. Ama şimdi 1T parametreli bu modeller bizim 100T parametreli beynimize çok yakın işler yapabiliyorlar. Yani bilgiyi bizden daha iyi temsil edebiliyorlar şu anda. Yakında …”
- “Plan yapamıyorlarsa, nasıl kod yazıyorlar? Yepyeni problemleri, çözümü olanlar kadar iyi çözemiyorlar. (Bu high-tech plagiarism [Chomsky]). Aynı şeyler IBM’in Jeopardy sistemi için de söylenmişti. Abartmayalım.”
- “Soru: Zorluk seviyesi çok daha fazla olan şeyleri yapabiliyorken daha az olanları yapamamanın, kolayca kandırılabilmelerinin açıklaması nedir?”
  - High-tech plagiarism, süper kopyacılar.
  - Bizler gibi hiyerarşik kabiliyet kümeleri yok. Başka türlü bir öğrenme bu. Sistematik, düzenli, hiyerarşik değil.
  - Başka ??
- “İnsanları tahmin edemiyorsan, insanları tahmin edilebilir hale getir. Bu tür araçlar insanların tahmin edilebilirliğini arttırır mı?”

15.1 Hinton'ın görüşü

Hoca Hinton'ın söylediğini şöyle aktardı: hayatı boyunca insan beyni kadar iyi çalışan sistemler yapmaya çalıştı; şimdi bir trilyon parametreli modeller var (hoca bir trilyon rakamının nereden geldiğini bilmediğini söyledi) (01:56:03). Beynimiz ise 100 trilyon parametrelidir: insan beyninde kabaca 100 milyar hücre olduğu ve her birinin ortalama 1000 bağlantısı olduğu düşünülür (ara adım: $100 \times 10^9 \times 1000 = 10^{14} = 100$ trilyon) (01:56:28). Hinton'a göre bu modeller ortalama bir insandan çok daha iyi sonuçlar verebiliyor; daha az parametreyle çok daha fazla bilgi tutabiliyorlar, yani bilgiyi temsil etme kabiliyetleri bizimkini geçmiş durumda. Bu da zekânın bir ölçüsüdür; belli bir noktadan sonra bizi aşacaklar ve böyle bir şeyi kontrol etmek kabaca mümkün değildir (01:56:56). Hoca spekülasyona girmek istemedi.

15.2 Chomsky'nin eleştirisi

Öteki taraftakiler "plan yapamıyorlar ama kod yazıyorlar" der: plan yapamayan bir şey nasıl kod yazar, yazdığı kodun nasıl bir anlamı olabilir? (01:57:49) Chomsky bu tür modeller için "high-tech plagiarism" terimini kullanır: süper kopyalayıcılar. Çok iyi kopyalıyorlar, kopyaladıklarını fark edemiyoruz bile, ama yalnızca kopyalıyorlar (01:58:12). Yeni bir şey üretme, yeni bir şey söyleme kabiliyetleri yok; şeyleri birbirine benzetiyorlar, ama bu benzetmenin menzili kısa. Çok farklı iki şeyi birbirine benzetip yepyeni bir birleşim üretemiyorlar (01:58:37). İnsanların böyle bir sisteme soracağı soruların kümesi çok büyük olmadığı için insanları tatmin edebiliyorlar.

Hoca "abartmayalım" uyarısını IBM'in Jeopardy sistemi örneğiyle bağladı: onun için de benzer şeyler söylenmişti ve hiçbiri tutmadı (01:59:05).

> **Ek bilgi (derste anlatılmadı):** IBM'in Jeopardy sistemi büyük olasılıkla Watson'dır. Watson, büyük olasılıkla 2011'de ABD'deki Jeopardy! bilgi yarışmasında insan şampiyonları yenmiş, ardından tıp gibi alanlarda devrim yaratacağı iddia edilmişti; bu beklentiler büyük ölçüde gerçekleşmedi.

15.3 Son not

Slayttaki son madde: "İnsanları tahmin edemiyorsan, insanları tahmin edilebilir hale getir." Hoca bunun ayrı bir hikâye olduğunu söyleyip dersi bitirdi (01:59:25).

Soru: Plan yapamamak kısmını biraz açabilir misiniz? Plandan kasıt bir problemin çözüm aşamaları mı? (01:59:49)

Cevap: Evet. Bir problemi çözmek için onu alt parçalara ayırmanız, her birini ayrı ayrı çözmeniz, birini çözemezseniz başka bir yolu denemeniz gerekir (02:00:00). Kabaca dinamik programlamadaki gibi bir yapı burada kurulamıyor; kurulsa bile o süreci temsil edip edemeyeceği belirsiz. Bunun temelinde, işlerin soldan sağa tek düze yürümesi vardır; geri dönme diye bir şey yoktur (02:00:38). Bir girdi verdiğinizde katman katman ilerler ve sonunda bir temsil oluşur. İşlerin yolunda gitmediğini fark ettiniz diyelim; bir önceki duruma dönmeniz gerekir. Ama bir önceki durum, kaldığınız yerdir ve oraya dönmek için modelle aynı büyüklükte başka bir yapının bir yerde tutulması gerekir (02:01:00). Geri dönülebilecek noktaların sayısı arttıkça bunlar üst üste biner; bunu seyrek (sparse) biçimde gerçekleştirebilecek yapılar gerekir ve şu an bu konuda başarılı işler yok (02:01:28).

Yine de kod yazabilmeleri bir muammadır: planlama yapmadan kod yazmak nasıl mümkün olabilir? Hocanın yorumu, bunun bizimkinden farklı bir öğrenme türü olduğudur (02:01:49). Bizim öğrenmemiz sistematik ve hiyerarşiktir. "Altı basamaklı sayıları çarpabiliyorum ama üç basamaklıları çarpamıyorum" diyen bir insanı yorumlayamayız (02:02:14). Burada ezberden söz edilmiyor: ezberlenemeyecek kadar büyük bir yapıyı yapabiliyor ama çok daha küçüğünü yapamıyor. Bu, onların öğrenme biçiminin bizimkiyle pek alakalı olmadığını gösteriyor (02:02:53).

Örnek Sorular

Soru 1 (ekrandan, 00:30:00; derste sözlü, 00:26:50): LLM'ler eğitim kümesinde 0 hataya erişebilirler mi? Düşük hataya erişebilmeleri için benzer anlama/kullanıma sahip kelimelerin tem-

silleri nasıl olmalıdır?

Çözüm (derste verildi): Yeterince büyük bir dil derleminde hayır. Aynı başlangıçla başlayan birçok cümle farklı devam eder; bu, aynı girdinin farklı sınıflara ait olduğu bir sınıflandırma problemidir ve her örnek için doğru token üretilemez. Hatayı azaltmak için model aynı bağlamda gelebilen farklı kelimelerin iç temsillerini birbirine yaklaştırmak zorundadır; bu yüzden benzer anlam ya da kullanıma sahip kelimelerin temsilleri benzeşir.

Soru 2 (ekrandan, 00:40:00): Zero-shot, one-shot ve few-shot ayarlarının farkı nedir? Hangisinde gradyan güncellemesi yapılır?

Çözüm (derste verildi): Zero-shot'ta modele yalnızca görev tanımı ve istem verilir ("Translate English to French: cheese =>"). One-shot'ta buna bir örnek ("sea otter => loutre de mer"), few-shot'ta birkaç örnek eklenir. Üçünde de gradyan güncellemesi yapılmaz; hepsi next token prediction ile eğitilmiş modele girdi olarak verilir. Gradyan güncellemesi yalnızca geleneksel fine-tuning'de yapılır.

Soru 3 (ekrandan, 00:50:00): Çoktan seçmeli sorular içeren bir veri kümesinde (örneğin CommonsenseQA: "What blocks sunshine?" {summer, park, desktop, sea, moon}) bir dil modeli X-shot ile nasıl değerlendirilir?

Çözüm (derste verildi): Her seçenek için, sorunun (ya da "The question is: … The answer is:" kalıbının) devamında o seçeneğin üretilme olasılığı hesaplanır. Beş olasılıktan en büyüğüne karşılık gelen seçenek modelin cevabı kabul edilir.

Soru 4 (ekrandan, 01:25:30): RL'de modele ne yapması gerektiği değil, yaptığının ne kadar iyi ya da kötü olduğu söylenir. Neden?

Çözüm (derste verildi): İki sebep vardır: o durumda yapılması gerekeni biz de doğrudan bilmiyor ya da tam emin olmayabiliriz; ayrıca tek bir doğru olmayabilir.

Soru 5 (derste sözlü, 00:54:10): Few-shot örneklerindeki etiketlerin hepsi aynı (örneğin hepsi pozitif) verilse model negatif bir cümleye yine "negatif" der mi?

Çözüm (derste verildi): Hoca bununla ilgili bir çalışma görmediğini, ama bunun pek mümkün görünmediğini söyledi; çünkü model örneklerde verilen etiketlerden birini söyler, onların dışında bir şey söylemez.

Soru 6 (ekrandan, 02:00:00; derste sözlü, 01:59:49): Zorluk seviyesi çok daha fazla olan şeyleri yapabiliyorken daha kolay olanları yapamamanın ve plan yapamamanın açıklaması nedir?

Çözüm (derste verildi): Slayttaki iki olası açıklama: modeller "high-tech plagiarism" yapan süper kopyacılardır; ya da bizimki gibi hiyerarşik kabiliyet kümeleri yoktur, sistematik ve hiyerarşik olmayan başka türlü bir öğrenme söz konusudur. Plan yapamamanın temelinde, işlemin soldan sağa tek çizgide ilerlemesi ve geri dönüp başka bir yol denemeyi sağlayacak bir yapının olmaması vardır.

Dersin Toparlaması

Bu derste büyük dil modellerinin nasıl eğitildiği ve uyarlandığı baştan sona izlendi. Next token prediction, ucuz ve ölçeklenebilir bir öz denetimli eğitim yoludur. Büyük bir dil derleminde sıfır hata imkânsız olduğu için model aynı bağlamda gelebilen kelimelerin temsillerini yakınlaştırır. Model yeterince büyüyünce, fine-tune edilmeden yalnızca görev tanımı ve birkaç örnekle (X-shot) yeni görevlerde başarılı olabildiği görüldü. Ama hatalar sistematik değildir ve bazı bulguların (karışık etiketlerle bile başarı) net bir açıklaması yoktur.

GPT-3'ün gelecek çalışmaya bıraktığı fine-tuning, InstructGPT ile RLHF biçiminde yapıldı: eğiticili fine-tuning, insan tercihleri, bu tercihleri taklit eden bir ödül modeli ve KL cezalı PPO ile pekiştirmeli öğrenme. Forum oyları gibi hazır tercih verileri ve Alpaca, OpenAssistant gibi açık kaynak çalışmalar bu süreci ucuzlatmaya çalışıyor. Son olarak bu modellerin henüz plan yapamadığı, Hinton ile Chomsky'nin karşıt görüşleri ve modellerin bizimkinden farklı, hiyerarşik olmayan bir biçimde öğreniyor olabileceği tartışıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.